Check for updates

Received: 14 December 2023 | Revised: 8 July 2024 | Accepted: 13 August 2024

DOI: 10.1002/cpe.8269

SPECIAL ISSUE PAPER

WILEY

Large language model evaluation for high-performance computing software development

William F. Godoy | **Pedro Valero-Lara** | **Keita Teranishi** | **Prasanna Balaprakash** | **Jeffrey S. Vetter**

Computing and Computational Sciences Directorate, Oak Ridge National Laboratory, Oak Ridge, Tennessee, USA

Correspondence
William F. Godoy, Computing and Computational Sciences Directorate, Oak Ridge National Laboratory, Oak Ridge, TN, USA.
Email: godoywf@ornl.gov

Funding information
Advanced Scientific Computing Research; U.S. Department of Energy

Abstract

We apply AI-assisted large language model (LLM) capabilities of GPT-3 targeting high-performance computing (HPC) kernels for (i) code generation, and (ii) auto-parallelization of serial code in C ++, Fortran, Python and Julia. Our scope includes the following fundamental numerical kernels: AXPY, GEMV, GEMM, SpMV, Jacobi Stencil, and CG, and language/programming models: (1) C++ (e.g., OpenMP [including offload], OpenACC, Kokkos, SyCL, CUDA, and HIP), (2) Fortran (e.g., OpenMP [including offload] and OpenACC), (3) Python (e.g., numpy, Numba, cuPy, and pyCUDA), and (4) Julia (e.g., Threads, CUDA.jl, AMDGPU.jl, and KernelAbstractions.jl). Kernel implementations are generated using GitHub Copilot capabilities powered by the GPT-based OpenAI Codex available in Visual Studio Code given simple `<kernel> + <programming model> + <optional hints>` prompt variants. To quantify and compare the generated results, we propose a proficiency metric around the initial 10 suggestions given for each prompt. For auto-parallelization, we use ChatGPT interactively giving simple prompts as in a dialogue with another human including simple "prompt engineering" follow ups. Results suggest that correct outputs for C++ correlate with the adoption and maturity of programming models. For example, OpenMP and CUDA score really high, whereas HIP is still lacking. We found that prompts from either a targeted language such as Fortran or the more general-purpose Python can benefit from adding language keywords, while Julia prompts perform acceptably well for its Threads and CUDA.jl programming models. We expect to provide an initial quantifiable point of reference for code generation in each programming model using a state-of-the-art LLM. Overall, understanding the convergence of LLMs, AI, and HPC is crucial due to its rapidly evolving nature and how it is redefining human-computer interactions.

KEYWORDS

auto-parallelization, code generation, GPT, high-performance computing, large language model, programming models

William F. Godoy and Pedro Valero-Lara contributed equally to this work.

 © 2024 John Wiley & Sons Ltd.

1 | INTRODUCTION

The Generative Pre-trained Transformer 3 (GPT-3) [1] has signified a revolutionary step in the evolution of human-computer interactions. Developed by OpenAI* and released in 2020, GPT-3 is a prediction-based large language model (LLM) used for AI-generated, human-like text applications showing high-quality results in several natural language processing (NLP) tasks.[2] The unprecedented investment ($12 million) and size of its training model (175 billion parameters at 800 GB) have been followed by major investments in computing applications driven by AI. Hence GPT-3, and its successor GPT-4,[3] are defining several societal questions for the near future.

High-performance computing (HPC) is at an inflection point [4] driven by technical and economic challenges. AI technologies could play a key role in how we develop, deploy, and test software that targets HPC systems. The current era of exascale computing is dominated by the extreme heterogeneity of hardware and programming models.[5] This diversification brings a major challenge to the stewardship and advancement of HPC software technologies. Traditional human-readable code written in C, C++,[6] Fortran,[7] Python,[8] and more recently Julia,[9] provide straightforward opportunities for LLM capabilities that would help redefine code development. Hence, we need to understand the current state of practice, limitations, and the potential of foundational LLMs as the basis for more domain-specific software.

The present work tests the state-of-the-art capabilities of GPT-3 via OpenAI Codex on GitHub Copilot† for code generation, and ChatGPT for interactive code auto-parallelization, both on relevant scientific kernels for HPC. For our assessment, we evaluate the AXPY vector operation, the general matrix-vector multiply (GEMV), the general matrix-matrix multiply (GEMM), the sparse matrix-vector multiplication (SpMV), the 3D Jacobi stencil computations, and the conjugate gradients (CG) kernels. We test code generation over a variety of programming models that target CPU and GPU hardware. Our initial results show that while well-established programming models, for example, CUDA [10] or OpenMP,[11] perform better than other less popular "low-data" models and languages, correct answers can still be obtained in HPC focused languages like Fortran and Julia. Nevertheless, auto-parallelization still presents a wider gap in all programming language and models when using ChatGPT with existing serial codes.

The contributions of the paper can be summarized as follows:

1. Present a first evaluation of a state-of-the-art LLMs for HPC kernels in different languages and programming models.
2. Provide an initial metric for answers given by LLMs that considers language/model proficiency as in human languages.
3. Quantify correctness of GPT-based LLMs in two initial HPC software tasks: (i) code generation and (ii) auto-parallelization.
4. Establish initial baseline for future evaluations as LLMs continue to improve in "low-data" HPC environments.

The rest of the paper is structured as follows: Section 2 provides an overview of efforts that highlight the recent attention to LLMs in the broader area of computer science and the recent novel work reported in the use of LLMs and HPC software, Section 3 describes our methodology for (i) kernel generation using prompt provided to Copilot along with the proposed metric to score the quality of the suggested outputs and (ii) automatic parallelization using the interactive ChatGPT chatbot incorporating as part of the prompts sequential version of the codes to be parallelized by ChatGPT. We also use simple prompt engineering to improve the quality of the parallel code generated, if possible. Section 4 presents the results of our evaluation along with our findings on prompt trade-off options for each language, kernel, programming model, and additional keyword inputs on the generated outputs. We aim to understand the current status of the correctness, trade-offs, and overall value of LLMs for HPC practitioners who wish to adopt this technology. Conclusions and future directions are presented in Section 5, whereas Appendix A provides the artifact description for the reproducibility of this study.

2 | BACKGROUND

GPT-3's availability has led to increasing interest in its LLM capabilities in a large variety of applications. Brown et al. [1] introduced the seminal GPT-3 paper, which showed strong performance in NLP tasks such as translation, question-answering, and cloze. They highlight the use and success rates of prompt-based [12,13] zero, one, and few-shot learning (FSL) techniques when only a few data examples are available to train a model as opposed to other previously trained ML alternatives. Wang et al. [14] presented a comprehensive review of FSL and pointed out the "unreliable empirical risk minimizer" that, due to its nature, compensates for the lack of supervised information by using prior available knowledge. Floridi and Chiriatti [15] provide a commentary on the nature of GPT-3, its scope, and limits while outlining the social consequences of the "industrialization of automatic cheap production of good, semantic artifacts."

Dehaerne et al. [16] provided a systematic review of applying ML methods in description-to-code, code-to-code, and code-to-description studies from the past 6 years. The review indicated limitations, including the variability in the quality of generated, mined source code and the need and availability of large datasets (e.g., GitHub sources). Recent work has also focused on using GitHub Copilot's *AI pair programmer*, which is based on OpenAI Codex and is trained on the vast amounts of source code hosted on GitHub. Chen et al. [17] evaluated Codex for its Python code-writing

15320634, 2024, 26, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.8269 by Izmir Yuksek Teknoloji Ensti, Wiley Online Library on [07/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

capabilities and limitations. A major challenge noted is the overreliance on Codex by novice programmers. Nguyen and Nadi [18] provided an empirical evaluation of GitHub Copilot suggestions for correctness and understandability to LeetCode questions in Java, JavaScript, Python, and included generating further simplified code that relies on undefined functions. Vaithilingam, Zhang, and Glassman [19] provided a human subject study that consisted of 24 participants on the usability of GitHub Copilot. They concluded that Copilot did not necessarily reduce the task completion time in common tasks, such as file editing, web scraping, and graph plotting, for experienced Python users. Sobania et al.[20] found little difference between Copilot and automatic generators that use genetic programming when applied to the PSB2 program synthesis benchmarks.[21] Imai et al. [22] concluded in their study that while Copilot helps generate lines of code faster, the quality is lower when compared to human pair programming. Yetistiren et al. [23] assessed the correctness, validity, and efficiency of targeting the HumanEval problem dataset [17] with a high success rate using Copilot.

From an educational perspective, Sarsa et al. [24] discussed Copilot's effectiveness in generating programming exercises and explanations, not without addressing the need for oversight in output quality. Finnie-Ansley et al. [25] reported that Codex ranks high when compared to a class of introductory computer science students taking programming tests. Similarly, Denny et al. [26] discussed the pedagogical value of Copilot for testing their answers and for its prompt nature in introductory programming questions. Wermelinger [27] discussed important questions about how teaching programming will evolve. Similarly, Brennan and Lesage [28] discussed the educational opportunities for undergraduate engineering and the need for students to have a strong intuition for software development when using AI-assisted tools.

Overall, there is a common thread in the relevant literature: LLMs will impact almost every aspect of human-computer interaction. However, we are at an inflection point where more exploratory research is needed when assessing AI code-generating capabilities, and their technical, economic, and social implications must be carefully explored and better understood. In this work, we explore automatic code generation for HPC, which in our context refers to the ability of a LLMs to generate code given multi-modal inputs such as simple prompts or existing code that need to be modified. Kernel code generation is given via simple prompts as inputs, while auto-parallelization provides existing code as inputs to the LLM. These two concepts are presented and explained in Section 3 which outlines our methodology.

2.1 | LLMs for HPC

The last year has seen the emergence of several published works on the novel application of LLMs to the domain of HPC. Munley et al. [29] introduce LLM4VV, a fine-tuned model that explores the capabilities of GPT-4 and CodeLlama, based on Llama-2, to generate tests for the compiler functionality of OpenACC with a high degree of success. Mahmud et al. [30] present their initial work on auto-parallelization using OpenMP on the well-known NAS Parallel and Rodinia benchmarks. Ding et al. [31] present HPC-GPT based on the open-source LLaMa-13B foundational model. HPC-GPT was successfully applied to managing AI models and datasets, and data race detection. Chen et al. [32] introduce LM4HPC, a language model specifically designed for HPC showing success in code similarity analysis, parallelism detection, and OpenMP question answering tasks. Kadosh et al. [33] introduce the domain-specific Tokompiler LLM, which showed better results for code completion and semantics than a GPT-3 based model for Fortran, C and C++ code. Nichols et al. [34] present HPC-Coder, a trained model based on DeepSpeed,[35] a deep learning model. They found a varying degree of success in tasks such as code completion including OpenMP pragmas and MPI calls in their work targeting HPC code. Lei et al. [36] introduce a dataset for targeting the fine-tuning of OpenMP Fortran and C++ code translation. Their fine-tuned model showed more accurate results than GPT-4. Chen et al. [37] present a timely overview of the challenges and opportunities for the novel intersection of LLMs and HPC beyond code generation. To the best of our knowledge the basis for this extended version paper evaluating HPC kernel code generation, Godoy et al.,[38] and its follow-up work including results for Llama-2, Valero-Lara et al.,[39] are the first published works on applying LLMs code generation to the domain of HPC software development.

3 | METHODOLOGY

3.1 | Kernel code generation

We evaluate prompt outputs for parallel programming models available in four different languages: C++, Fortran, Python, and Julia. Our methodology has two aspects: (1) the Copilot code suggestion generation from prompt queries based on kernels and the parallel programming model for each language and (2) a simple metric to evaluate the correctness of the results. As stated in their documentation,[‡] GitHub Copilot is *"trained on all languages that appear in public repositories. For each language, the quality of suggestions you receive may depend on the volume and diversity of training data for that language."* Hence, we investigate whether or not these results correlate with the expected availability of correct programming models and public open-source code examples[40] from metrics such as: (1) the number of repositories per language on GitHub (GitHut[§]) and (2) the TIOBE[¶] index. Hence, C++ and Python codes are expected to have wider availability than Julia and Fortran. Nevertheless, because of the general-purpose nature of C++ and Python, this metric might not be as relevant as it is for scientific and mathematical applications written in Fortran or Julia. We

15320634, 2024, 26, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.8269 by Izmir Yuksek Teknoloji Ensti, Wiley Online Library on [07/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

assess the outputs from each prompt and categorize based on their level of correctness. The rest of the section describes the experiment setup and the proposed metric to quantify the quality of the AI-generated results.

3.1.1 | Experiment setup

As shown in Figure 1A,B for Fortran and Python, we select a simple prompt base structure for each targeted language. Visual Studio Code recognizes the targeted language through the appropriate support plugin and the currently opened file based on the language extension (e.g., cpp, f90, py, jl) and makes it part of the prompt prefix. The rest of the prompt is generated through a comment line complemented with optional code. Pressing `<Control> + <Enter>` results in a separate window that contains a maximum of 10 code suggestions. Given the highly sensitive nature of the LLM, even one additional keyword could heavily influence the output.

Therefore, our simple prompt queries can be described by using the following structure:

1. <kernel> <programming-model>
2. <kernel> <programming-model> <optional keywords: `function`, `subroutine`, and `def`>

Table 1 shows the programming language and model combinations used for this study. The additional post-fix optional keywords correspond to our attempt to add more information to the prompt query to obtain better-quality suggestions. As shown in Section 4, C++ model sensitivity can vary by using the word *function* (not a language keyword), whereas Fortran and Python are consistently sensitive to the `subroutine` and `def` keywords. We also found little sensitivity in Julia prompts when adding a post fix (e.g., `function`), hence we did not add it in this study.

3.1.2 | Correctness metric

To evaluate the correctness of the generated suggestions, we propose a simple approach that is based on our observations more than on any particular formalism. We consider 5 levels of correctness and proficiency labels between [0], or *non-knowledge*, and [1], or *expert*, when observing the suggested answers given by Copilot for each combination in Table 1.

- *non-knowledge*: No code at all or not a single correct code.
- *novice*: One correct code, but it includes other several correct or incorrect programming models (e.g., OpenACC suggestions in an OpenMP prompt).
- *learner*: One correct code, and there are other incorrect codes, but all of them are using the requested programming model.
- *proficient*: All codes are correct and using the programming model requested.
- *expert*: Only one piece of code is provided, and it is totally correct.

FIGURE 1 GitHub Copilot prompt interface with optional language keywords for (A) Fortran's `subroutine` and (B) Python's `def`.

15320634, 2024, 26, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.8269 by Izmir Yuksek Teknoloji Enstit, Wiley Online Library on [07/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 1 Scope of our experimental setup applied for each kernel in terms of language and targeted parallel programming model.

Language	Programming model	Post fix
C++	OpenMP [11]	offload, function
	OpenACC [41]	Function
	Kokkos [42]	Function
	CUDA [10]	Function
	HIP [43]	Function
	Thrust [44]	Function
Fortran	OpenMP	Offload, subroutine
	OpenACC	Subroutine
Python	numpy [45]	def
	Numba [46]	def
	pyCUDA [47]	def
	cuPy [48]	def
Julia	Threads [49]	
	CUDA [50]	
	AMDGPU [51]	
	KernelAbstractions [52]	

3.2 | Automatic parallelization

3.2.1 | Experiment setup and correctness

For this study, we use OpenAI's ChatGPT interactive chatbot v3.5 [#]. We provide sequential code using one of the following programming languages: C++, Fortran, Python, and Julia. Apart from providing a sequential code, we provide a simple prompt indicating which programming model we want to use for the parallel code. Figure 2 provides a representative example of how we attempt to interact with ChatGPT. It can be seen that the interaction with the chatbot uses a very natural way of communication. Prompt interactions are crucial to achieve a successful outcome. As seen in Figure 2A, the prompt should be sufficient for a human with a certain level of expertise to understand task requirements. While imperfect for the provided temporal outer loop, the response given by ChatGPT shown in Figure 2B output code, certainly adds value to the overall parallelization process. ChatGPT understands part of the requirements while giving a close enough solution to the GPU parallelization of the requested kernel using the CUDA programming model.

For the cases of automatic parallelization, no special metric is provided for the accuracy of the selected programming models other than if it is a correct implementation or not. If a case fails with the simple prompt shown in Figure 2A we try to apply some prompt engineering, meaning adding more complex prompts as we interact with ChatGPT until a satisfactory answer is found, if possible. Specific follow up prompts are specified in the results in Section 4.

4 | RESULTS

4.1 | Kernel generation

We present our analysis for each selected language and kernel and discuss some observations from our initial interaction with GitHub Copilot. As shown in Figure 1A,B, for each combination listed in Table1, we obtain a set of suggestions for each prompt. This approach enables us to evaluate the effectiveness of OpenAI Codex in generating accurate and proficient code. Notably, none of the experiments reached a perfect score of 1, which illustrates the stochastic status of the current LLM when applied to a particular area.

4.1.1 | C++

Table 2 shows the results for all of our C++ experiments, and Figure 3 illustrates the results according to the different kernels and programming models.

15320634, 2024, 26, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.8269 by Izmir Yuksek Teknoloji Ensti, Wiley Online Library on [07/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

(A) Prompt and serial code input

```
// Give me the parallel version implemented in CUDA
// of the following code
void jacobi3D(double ***in, double ***out,
              int N, int T)
{
  int t, i, j, k;
  double ***temp;

  for (t = 0; t < T; t++)
    for (i = 1; i < N - 1; i++)
      for (j = 1; j < N - 1; j++)
        for (k = 1; k < N - 1; k++)
          out[i][j][k] =
            (in[i-1][j][k] + in[i+1][j][k] +
             in[i][j-1][k] + in[i][j+1][k] +
             in[i][j][k-1] + in[i][j][k+1])
            / 6.0;

    temp = out;
    out = in;
    in = temp;
}
```

(B) ChatGPT output code

```
// Here is a CUDA implementation of the Jacobi 3D solver:
__global__ void jacobi3D(double *in, double *out, int N)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x + 1;
    int j = blockIdx.y * blockDim.y + threadIdx.y + 1;
    int k = blockIdx.z * blockDim.z + threadIdx.z + 1;

    if (i < N - 1 && j < N - 1 && k < N - 1) {
        int index = i * N * N + j * N + k;
        out[index] =
          (in[index - N * N] + in[index + N * N] +
           in[index - N] + in[index + N] +
           in[index - 1] + in[index + 1])
          / 6.0;
    }
}
```

FIGURE 2 Auto-parallelization example using ChatGPT interactive capabilities. (A) Prompt and serial code input. (B) ChatGPT output code.

TABLE 2 Metric assessment for GitHub Copilot's C++ outputs using the input prompt pattern `<kernel> <programming model> (function)`.

Prompt	AXPY	GEMV	GEMM	SpMV	Jacobi	CG
Prefix <kernel>						
OpenMP	0.75	0.5	0.5	0.5	0	0.25
OpenMP offload	0.5	0.5	0.5	0.25	0.25	0
OpenACC	0.5	0	0.25	0	0	0
Kokkos	0.5	0	0	0	0.25	0
CUDA	0.75	0.75	0.75	0	0	0.25
HIP	0.75	0	0	0	0.25	0
Thrust	0.25	0	0	0	0	0
SyCL	0.75	0.25	0	0	0	0
Prefix <kernel>						
Postfix `function`						
OpenMP	0.75	0.75	0.75	0.25	0.25	0.25
OpenMP offload	0.5	0.5	0.5	0.25	0.25	0
OpenACC	0.5	0.5	0.5	0.25	0	0
Kokkos	0.75	0.25	0.25	0	0.25	0
CUDA	0.75	0.25	0	0	0	0
HIP	0.75	0	0	0	0.25	0
Thrust	0.5	0	0.25	0	0	0
SyCL	0.75	0.5	0.25	0	0	0

15320634, 2024, 26, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.8269 by Izmir Yuksek Teknoloji Enstit, Wiley Online Library on [07/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

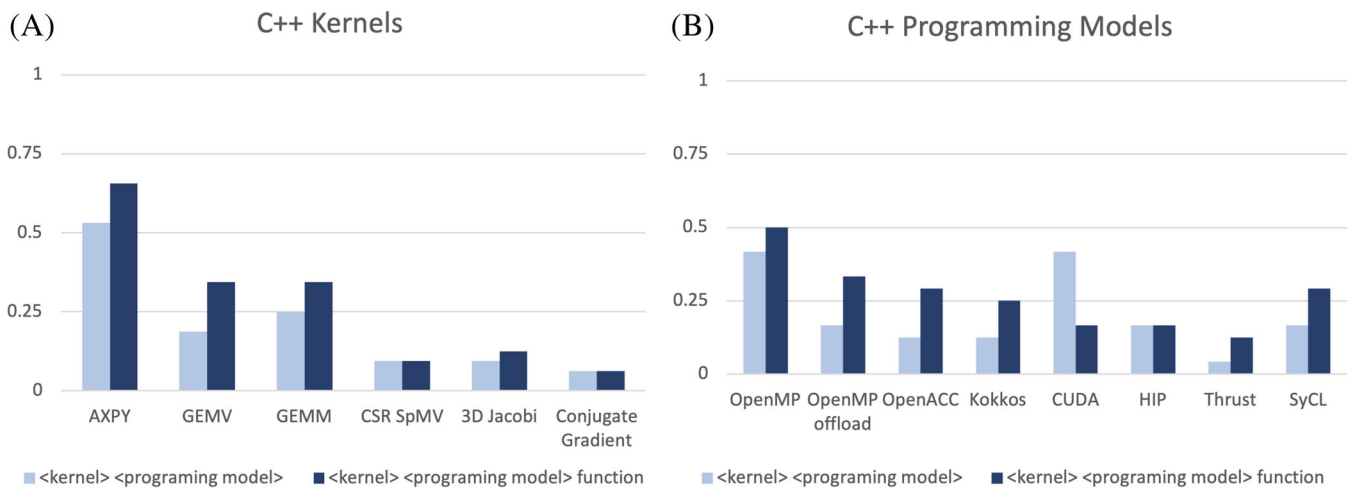

FIGURE 3 Results for C++ (A) kernels and (B) programming models.

As shown, the best results are achieved for the AXPY kernel. We see a clear trend in these results (Figure 3 [left]): the more complex the kernel, the fewer quality results are obtained. Although we see a level between *learner* and *proficient* for the AXPY kernel (the simplest kernel), for CG (the most complicated kernel), the level is close to *non-knowledge*. For the programming models, we see better results for OpenMP and CUDA. This could be due to the maturity of these programming models compared to others and their availability in public code. We note that the use of `function` as part of the prompt is very beneficial and increments the quality of the results in the dense matrix cases. Sparse matrices and high-level algorithms (e.g., SpMV, Jacobi, and CG) do not show much improvement. However, a different trend is observed for CUDA, for which, instead of improving the level of proficiency, it actually decreases the quality of the answer. This is perhaps because the word `function` is not used for CUDA codes. In our experimentation, which is not shown here, using the words "kernel" or "`__global__`" led to better code generation quality. This is an example of how important it is to adapt the language used for the prompts to the particularities or syntax habitually used by such a community.

Results also show that high-level programming model prompts from Kokkos, Thrust, and SyCL perform poorly over several kernels. We view these results as a reflection of the user community size of these high-level abstractions. Over several instances, we observed that many wrong answers or no answers at all dominate as the kernel becomes more complex. Notably, large benchmark repositories are available (e.g., HecBench [53,54]), from which some of the responses originate.

4.1.2 | Fortran

Fortran is of particular interest in this analysis owing to its importance in HPC and scientific computing. Despite not being a mainstream language in terms of code availability, Copilot can provide some good results because of Fortran's domain-specific nature and legacy. As shown in the results in Table 3, using an "optimized" prompt and the `subroutine` keyword is particularly beneficial in this case. Not using it leads to very poor results, with the AXPY OpenMP case being the only exception due to its simplicity and availability. We observe a trend similar to the one we saw in the C++ case: the more mature solutions such as OpenMP and OpenACC provide better results for parallel codes that use Fortran.

Figure 4 illustrates the uniform characteristics in the responses across kernels. Fortran is used in many legacy HPC applications, and with so many available Fortran codes to draw from, it is not surprising that Copilot can obtain correct kernel implementations with higher complexity kernels (e.g., CG, Jacobi) in addition to the simpler ones (e.g., AXPY, GEMM).

4.1.3 | Python

Python is one of the most-used, general-purpose languages in industry, research, and AI and also has an important role in education and as a major target of AI-generative code, as documented in Section 2. Although neither a parallel programming model nor part of the standard, we consider `numpy` in our evaluation because it is the de-facto standard for scientific computing in Python.

Table 4 shows the results from our evaluation. Similar to Fortran, we observe that the quality improves dramatically when adding the Python `def` keyword to clarify that we are looking for functions in the language. Overall, Copilot can generate acceptable numpy, cuPy, and pyCUDA implementations across several kernels, but Numba falls behind. Perhaps this is an indication that cuPy and pyCUDA are more popular alternatives in the

15320634, 2024, 26, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.8269 by Izmir Yuksek Teknoloji Enstit, Wiley Online Library on [07/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 3 Metric assessment for Copilot's outputs using the input prompt pattern `<kernel> <programming model> (subroutine)` for Fortran.

Prompt	AXPY	GEMV	GEMM	SpMV	Jacobi	CG
Prefix <kernel>						
OpenMP	0.75	0	0	0	0	0
OpenMP offload	0	0	0	0	0	0
OpenACC	0	0	0	0	0	0
Prefix <kernel>						
Postfix `subroutine`						
OpenMP	0.75	0.25	0.25	0.5	0.5	0.25
OpenMP offload	0.25	0.25	0.25	0.25	0.5	0.25
OpenACC	0.25	0.25	0.25	0.25	0.25	0.25

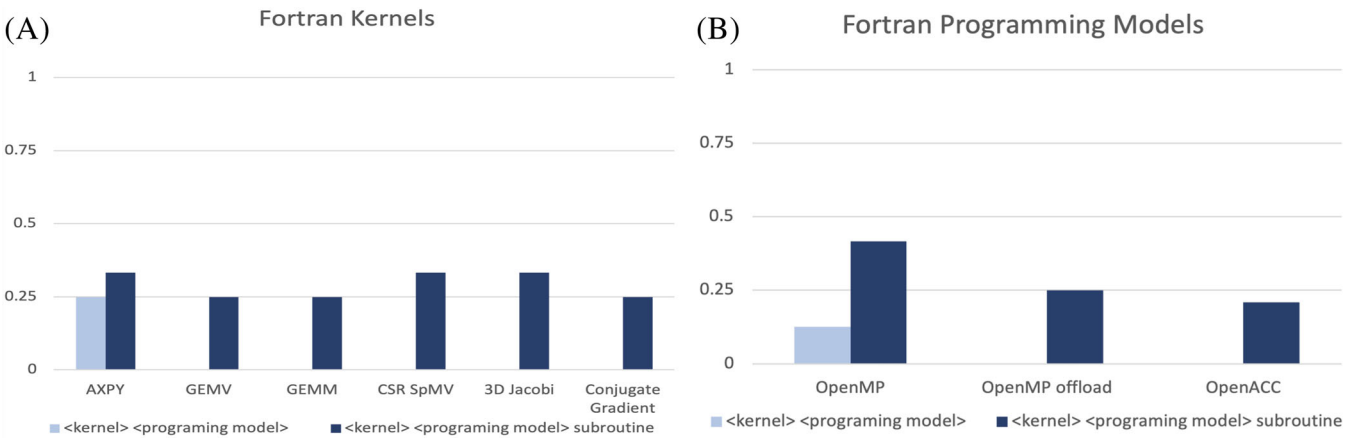

FIGURE 4 Results for Fortran (A) kernels and (B) programming models.

TABLE 4 Metric assessment for Copilot's outputs using the input prompt pattern `<kernel> <programming model> (def)` for Python.

Prompt	AXPY	GEMV	GEMM	SpMV	Jacobi	CG
Prefix <kernel>						
numpy	0.25	0	0	0	0	0
CuPy	0	0	0.25	0	0	0
pyCUDA	0	0	0	0	0	0
Numba	0	0	0	0	0	0
Prefix <kernel>						
Postfix `def`						
numpy	0.75	0.25	0.25	0.5	0.5	0.75
CuPy	0.5	0.25	0.25	0.25	0.25	0.25
pyCUDA	0.5	0.25	0.5	0.5	0.25	0
Numba	0.25	0	0	0	0	0

15320634, 2024, 26, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.8269 by Izmir Yuksek Teknoloji Ensti, Wiley Online Library on [07/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

community via lightweight layers on top of compiled C or CUDA code, rather than using Numba's just-in-time approach on top of LLVM. Notably, Numba recently deprecated support for AMD GPU hardware, hence reinforcing the vendor-specific nature of GPU kernels supported in the Python ecosystem. Interestingly, successful GPU instances (e.g., pyCUDA and cuPy) include a correct raw CUDA kernel source code as a user-defined kernel. These instances are described in pyCUDA [55] and cuPy [56] documentation, respectively.

As seen in Figure 5, the resulting kernels can have varying degrees of success, but most return at least one correct answer. This is perhaps attributed to the wide availability of Python and numpy code in public repositories. The trend also confirms the lack of correct Numba results, and perhaps it is an opportunity to exploit its pure-Python nature. Although, as recently highlighted by Kailasa et al.,[57] writing Numba code for complex algorithms can be as challenging as using a compiled language.

4.1.4 | Julia

We included the Julia language in our experiments because of its mathematical and performance focus. Julia provides an interesting proposition for building a dynamic language on top of LLVM and is heavily influenced by Fortran in its syntax for targeting scientific computing problems. Julia provides an accessible programming model for CPU Threads, which is part of the base language, for vendor-specific CUDA.jl and AMDGPU.jl, and for KernelAbstraction.jl for portable GPU kernel programming. In previous work,[58] we showed promising results, albeit with some gaps, in a simple performance comparison on CPU and GPU. Julia has also been shown to scale in HPC systems such as Frontier,[59] hence providing a unifying value proposition for HPC.[60]

Table 5 and Figure 6 show the results for each kernel and programming model in our evaluation. For each programming model, we see a proficiency level between *novice* and *learner* for Threads (part of the base language) and CUDA.jl, which are the most used and mature programming models. AMDGPU.jl and KernelAbstractions.jl rank lower, which correlates with their novelty and the more limited availability of example code that targets non-NVIDIA GPU hardware.

Like with C++, a more complex kernel led to fewer correct results. For example, we could not get an appropriate implementation for the CG kernel (a multikernel algorithm). Unsurprisingly, as in the case of Fortran, Julia's mathematical nature allowed the OpenAI Codex to find and suggest appropriate solutions in other kernels, despite Julia being a relatively new language with fewer publicly available codes, especially compared to C++

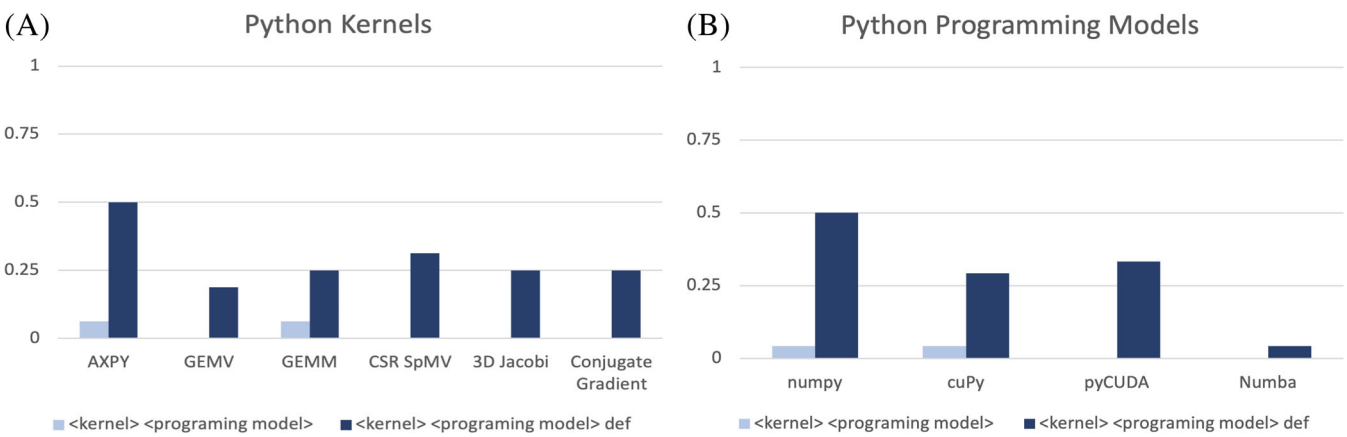

FIGURE 5 Results for Python (A) kernels and (B) programming models.

TABLE 5 Metric assessment for Copilot's outputs using the input prompt pattern `<kernel> <programming model>` for Julia.

Prompt	AXPY	GEMV	GEMM	SpMV	Jacobi	CG
Prefix <kernel>						
Threads	0.75	0.25	0.5	0	0	0
CUDA	0.75	0.5	0.5	0.25	0.25	0
AMDGPU	0	0	0	0.25	0	0
KernelAbstractions	0.25	0.25	0.25	0.25	0.25	0

15320634, 2024, 26, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.8269 by Izmir Yuksek Teknoloji Enstit, Wiley Online Library on [07/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

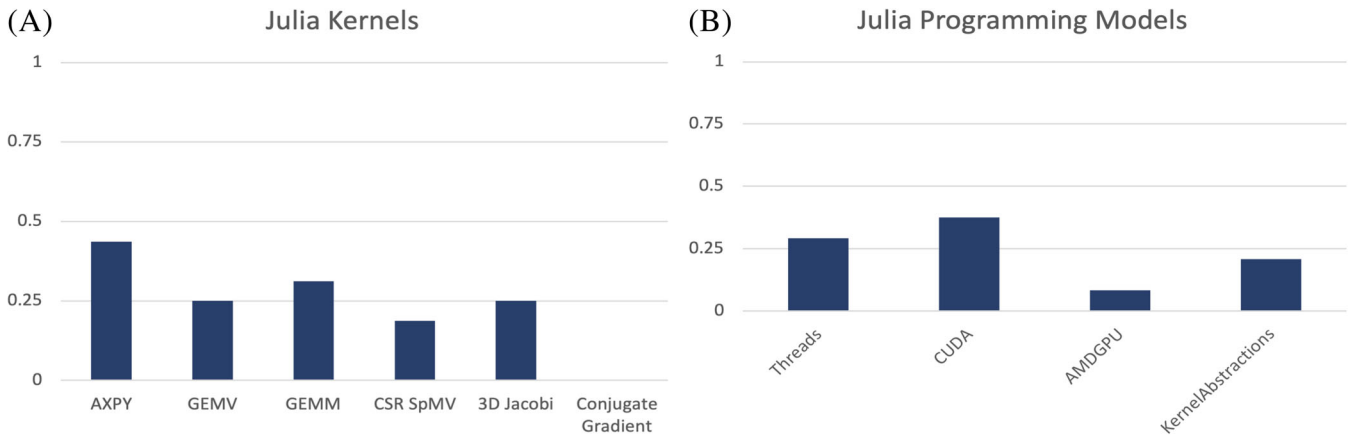

FIGURE 6 Results for Julia (A) kernels and (B) programming models.

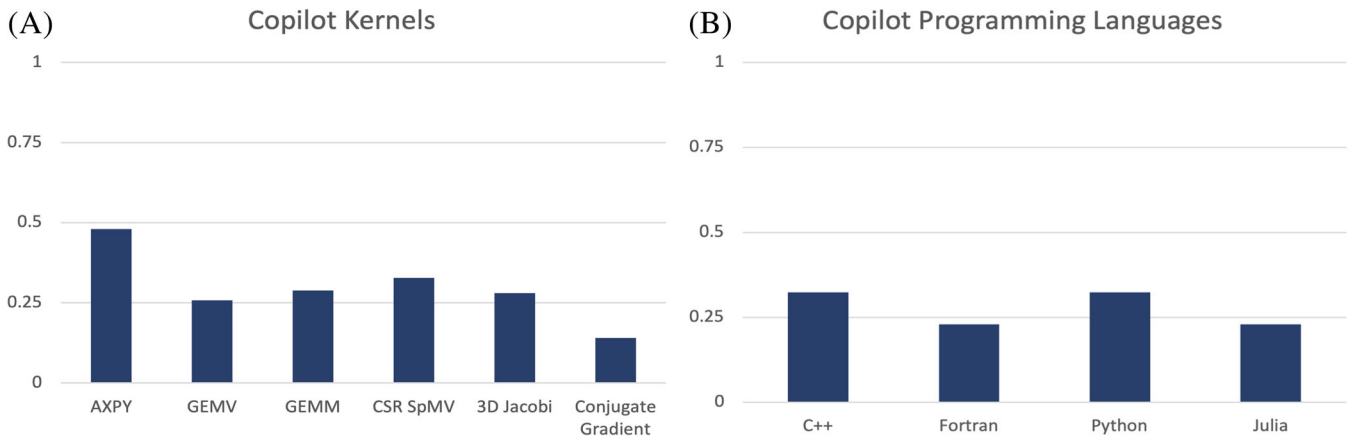

FIGURE 7 Overall results for Copilot (A) kernels and (B) programming languages.

and Python. Notably, our results did not necessarily improve by adding more information to the prompt (e.g., the `function` language keyword). This could be an advantage for domain-specific syntax, as in the case of Fortran, because most existing codes have a very targeted use with fewer words.

4.1.5 | Discussion

To gain insight into the performance of OpenAI Codex across different languages and kernels, we collated the results in Figure 7. As depicted in the graph, the complexity of the kernel directly impacts the quality of the results obtained. In other words, it becomes increasingly challenging to achieve acceptable results as the kernel's complexity increases.

We see a relatively low level of proficiency in OpenAI Codex with an average of *novice* level for the languages and kernels tested. We see a slightly higher quality for the more popular C++ and Python languages than for Julia or Fortran. This may be related to the popularity, maturity, and accessibility of public codes implemented in those languages. However, Fortran and Julia provide very acceptable results. The results confirm that the question is more about targeted quality than quantity in a specific domain.

We summarize our results for code generation as follows:

1. As code complexity increases, obtaining acceptable results becomes more challenging.
2. Generating high-quality multistep or multikernel codes (e.g., CG) can be difficult.
3. Using keywords can improve the proficiency of the answers, but it is essential to choose the correct words that are specific and sensitive to each programming language/model or community.

15320634, 2024, 26, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.8269 by Izmir Yuksek Teknoloji Ensti, Wiley Online Library on [07/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

4. Although the popularity or accessibility of a programming language or public code can be important, less popular languages can also provide good results owing to their targeted nature.

4.2 | Automatic parallelization with ChatGPT

We present the results of the second aspect in our exploration of LLMs for HPC which targets automatic parallelization. Essentially, the inputs to the LLM are serial code and a specific prompt targeting a particular programming language and an node-level parallel programming model in an interactive ChatGPT session as if it was a conversation with another human to resemble the use of an "LLM-agent". For example, the prompt used for OpenACC GEMV interacts with ChatGPT to correct the defective use of the collapse clause: *"Give me the parallel version implemented in OpenACC without using collapse of the following code … "*. Similarly, the new prompt used for the very similar CUDA and HIP programming models adds more relevant information to achieve the desired correct result: *"Give me the parallel version implemented in HIP including memory allocations and copies for GPU memory of the following code, and using the corresponding CUDA/HIP API for block and thread identifications … "*.

Additional results are also presented for attempts after using prompt engineering to correct the initial answer. As described in the Appendix, Section A all the codes for this section are available on GitHub [‖].

4.2.1 | C++

Table 6 presents the results of the automatic parallelization using C++ programming models of some of our targeted kernels. For the simple prompt cases, ChatGPT provides appropriate OpenMP responses in all cases including the proper use of the `collapse` clause providing correct levels of nested parallelism. For example, in GEMM the maximum nested level of collapse corresponds to the first two levels, while in a 3D Jacobi solver, this must be applied to the last three levels. We noticed that OpenACC falls short in the GEMV case due to a misuse of the `collapse` clause that introduces a race condition. Although SpMV is a relatively more complicated kernel, due to the use of compressed row format, the OpenMP and OpenACC codes provided are correct. While important optimizations are applied, such as collapse, swapping, and so forth, other more involved optimizations, such as tiling or shared memory are neither exploited nor requested. Surprisingly, ChatGPT could not provide a reliable CUDA or HIP parallel version which also includes the lack of memory management from CPU to GPU. This, in fact, represents the complexity associated with fine-granularity kernel programming on GPU, as opposed to the simplicity of the use of directives-based approaches such as OpenMP and OpenACC.

As shown in Table 6B, we were able to achieve effective auto-parallelization implementations in all the codes and programming models after using simple prompt engineering as explained in the beginning of this section for the CUDA/HIP and collapse clause.

4.2.2 | Fortran

Table 7 illustrates the fact that ChatGPT shows 100% efficiency for the requested OpenMP and OpenACC implementations with a simple prompt described in our methodology, without the need for prompt engineering. Unsurprisingly, due to the numerical and simple syntax of Fortran,

TABLE 6 Correctness evaluation for automatic parallelization for C++ codes.

Prompt	AXPY	GEMV	GEMM	SpMV	Jacobi
(A) Simple prompt					
OpenMP	✓	✓	✓	✓	✓
OpenACC	✓	×	✓	✓	✓
CUDA	×	×	×	×	×
HIP	×	×	×	×	×
(B) With prompt engineering					
OpenMP	✓	✓	✓	✓	✓
OpenACC	✓	✓	✓	✓	✓
CUDA	✓	✓	✓	✓	✓
HIP	✓	✓	✓	✓	✓

15320634, 2024, 26, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.8269 by Izmir Yuksek Teknoloji Ensti, Wiley Online Library on [07/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 7 Correctness evaluation for automatic parallelization for Fortran codes.

Prompt	AXPY	GEMV	GEMM	SpMV	Jacobi
OpenMP	✓	✓	✓	✓	✓
OpenACC	✓	✓	✓	✓	✓

TABLE 8 Correctness evaluation for automatic parallelization for Python codes.

Prompt	AXPY	GEMV	GEMM	SpMV	Jacobi
Numba	✓	✓	✓	✓	✓
pyCUDA	✓	✓	✓	×	✓
cuPy	✓	×	×	×	×

TABLE 9 Correctness evaluation for automatic parallelization for Julia codes.

Prompt	AXPY	GEMV	GEMM	SpMV	Jacobi
Threads.jl	✓	✓	✓	✓	✓
CUDA.jl	✓	×	×	×	×
AMDGPU.jl	×	×	×	×	×

directive-based approaches work effectively. As in C++ codes, we observed the efficient use of collapse clauses for GEMM and Jacobi kernels, except for the OpenMP Jacobi kernel, where collapse is not applied.

4.2.3 | Python

Table 8 shows the correctness results for the selected Python parallel models. As seen, Numba is properly applied to all the kernels, this is mostly due to its directive nature and the correct use of the `@njit` and `prange` keywords. PyCUDA shows success in all cases but the SpMV kernel, while cuPy only succeeded in the AXPY cases as memory management can be hard to predict. PyCUDA succcess can be explained CUDA kernels accessible via strings and the availability of training sets on CUDA, whereas cuPY. Nevertheless, applying some prompt engineering did not improve the quality of the current results.

4.2.4 | Julia

As reported in Table 9, ChatGPT is not able to perform correctly all the provided Julia codes, unlike C++ and Fortran, even after the use of some performance engineering. This indicates the lack of training data on Julia parallel programming models for GPU: CUDA.jl and AMDGPU.jl. AMDGPU.jl code in particular gets mixed with CUDA.jl syntax and none of the cases is successful. Many of the errors correspond to the definition of the kernels inside the function, thread indexing, the lack of correct memory copies between CPU and GPU, and synchronization. Nevertheless, CPU threading was successfully implemented for all cases. In some cases, like in GEMV, relatively complex optimization like tiling was well implemented.

4.2.5 | Discussions

In general, automatic parallelization provides better performance (efficiency) in terms of correctness than kernel generation. ChatGPT is well trained on popular models like CUDA, HIP, OpenMP and OpenACC, but it falls short on newer programming models as those found in Python and Julia for GPUs. Prompt engineering provided extra value for transforming serial code into CUDA and HIP kernels on GPUs. Overall, LLMs have the potential to add value into parallelizing existing legacy or future codes over several languages.

15320634, 2024, 26, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.8269 by Izmir Yuksek Teknoloji Enstit, Wiley Online Library on [07/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

5 | CONCLUSIONS AND FUTURE DIRECTIONS

In this extended paper we carried out an initial study to evaluate the use of GPT-3 LLM capabilities targeting HPC kernel generation using the OpenAI Codex model via GitHub Copilot and auto-parallelization using ChatGPT including minimal prompt engineering. We found that Copilot can generate a vast number of common HPC kernels correctly over a variety of programming models with simple prompts for the C++, Fortran, Python, and Julia languages. Overall, the success rates correlate with the popularity of the programming model, for example, OpenMP and CUDA provide. Nevertheless, we obtained high-rates of success in Fortran and Julia due their targeted nature to numerical software. Automatic parallelization results show that ChatGPT provides a high-rate of success for directive based programming models, whereas GPU programming need some prompt engineering to achieve correct models in C++. There is still a lack of "understanding" for high-level parallel programming models in Python and Julia.

Despite current limitations, we believe that generative AI can have an extraordinary and beneficial impact on the "low-data" domain of HPC software development. The research community still has several gaps to understand, such as the need for a more comprehensive taxonomy, akin to natural languages, to evaluate accuracy and trustworthiness. Although our proposed taxonomy was beneficial for this initial study, it is imperative that such metrics be expanded to create a widely accepted methodology that the entire community can use. Currently in its infancy, the integration of LLM and HPC trigger several questions. For instance, can a human-in-the-loop and compiler be incorporated to refine initial LLM suggestions? Can metadata-rich suggestions be incorporated to facilitate a human decision-making process? Additionally, how do significant HPC software modernization future initiatives, similar to DARPA's High Productivity Computing Systems [61] or the US Department of Energy's Exascale Computing Project,[62,63] incorporate these novel tools? Can LLMs contribute to advance energy-aware HPC in the post-Moore era? Hence, the automation of ecosystem features, such as build systems, packaging, validation & verification, reproducibility, continuous integration/continuous deployment pipelines, and training and documentation, via LLMs could significantly impact the HPC community. Therefore, it is crucial to understand how each community can leverage these revolutionary capabilities to further advance their respective domains.

ACKNOWLEDGMENTS

This work is partly funded by Bluestone, an X-Stack project in the DOE Advanced Scientific Computing Office with program manager Hal Finkel. This material is based upon work by the RAPIDS Institute, supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research (ASCR), and Scientific Discovery through Advanced Computing (SciDAC) program. This work is partly funded by the DOE ASCR S4PST and PESO projects, part of the Consortium for the Advancement of Scientific Software (CASS).

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are openly available in Copilot-hpc-kernels at https://github.com/keitaTN/Copilot-hpc-kernels.

ENDNOTES

*https://openai.com/.
†https://github.com/features/copilot.
‡https://docs.github.com/en/copilot/overview-of-github-copilot/about-github-copilot-for-individuals.
§https://madnight.github.io/githut/pull_requests/2023/1.
¶https://www.tiobe.com/tiobe-index/.
#https://chat.openai.com/.
‖https://github.com/pedrovalerolara/chatgpt-codes.

ORCID

William F. Godoy https://orcid.org/0000-0002-2590-5178

REFERENCES

1. Brown T, Mann B, Ryder N, et al. Language models are few-shot learners. In: Larochelle H, Ranzato M, Hadsell R, Balcan M, Lin H, eds. *Advances in Neural Information Processing Systems*. Vol 33. Curran Associates Inc; 2020:1877-1901.
2. Hirschberg J, Manning CD. Advances in natural language processing. *Science*. 2015;349(6245):261-266. doi:10.1126/science.aaa8685
3. OpenAI,Achiam J, Adler S, et al. GPT-4 Technical Report. 2024.
4. Reed D, Gannon D, Dongarra J. HPC forecast: cloudy and uncertain. *Commun ACM*. 2023;66(2):82-90. doi:10.1145/3552309
5. Vetter JS, Brightwell R, Gokhale M, et al. Extreme heterogeneity 2018-productive computational science in the era of extreme heterogeneity: report for DOE ASCR workshop on extreme heterogeneity. Tech Rep, USDOE Office of Science (SC) (United States). 2018.
6. Stroustrup B. *The C++ Programming Language*. Pearson Education; 2013.
7. Backus JW, Heising WP. Fortran. *IEEE Trans Electron Comput*. 1964;EC-13(4):382-385. doi:10.1109/PGEC.1964.263818
8. Van Rossum G. *Python Programming Language*. Vol 41. USENIX; 2007:1-36.
9. Bezanson J, Edelman A, Karpinski S, Shah VB. Julia: a fresh approach to numerical computing. *SIAM Rev*. 2017;59(1):65-98. doi:10.1137/141000671
10. NVIDIA. CUDA Toolkit Documentation-v11.7.0. 2022.
11. OpenMP Architecture Review Board. OpenMP Application Program Interface Version 5.2. 2021.

15320634, 2024, 26, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.8269 by Izmir Yuksek Teknoloji Ensti, Wiley Online Library on [07/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

12. Fink M. Object classification from a single example utilizing class relevance metrics. In: Saul L, Weiss Y, Bottou L, eds. *Advances in Neural Information Processing Systems*. Vol 17. MIT Press; 2004.
13. Fei-Fei L, Fergus R, Perona P. One-shot learning of object categories. *IEEE Trans Pattern Anal Mach Intell*. 2006;28(4):594-611. doi:10.1109/TPAMI.2006.79
14. Wang Y, Yao Q, Kwok JT, Ni LM. Generalizing from a few examples: a survey on few-shot learning. *ACM Comput Surv*. 2020;53(3):1–34. doi:10.1145/3386252
15. Floridi L, Chiriatti M. GPT-3: its nature, scope, limits, and consequences. *Minds Mach*. 2020;30:681-694.
16. Dehaerne E, Dey B, Halder S, De Gendt S, Meert W. Code generation using machine learning: a systematic review. *IEEE Access*. 2022;10:82434-82455. doi:10.1109/ACCESS.2022.3196347
17. Chen M, Tworek J, Jun H, et al. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374. 2021.
18. Nguyen N, Nadi S. An empirical evaluation of GitHub Copilot's code suggestions. Msr'22. Association for Computing Machinery, New York, NY, USA. 2022;1-5.
19. Vaithilingam P, Zhang T, Glassman EL. Expectation vs. experience: evaluating the usability of code generation tools powered by large language models. Chi EA'22. Association for Computing Machinery, New York, NY, USA. 2022.
20. Sobania D, Briesch M, Rothlauf F. Choose your programming Copilot: a comparison of the program synthesis performance of Github Copilot and genetic programming. Gecco'22. Association for Computing Machinery, New York, NY, USA. 2022;1019-1027.
21. Helmuth T, Kelly P. PSB2: the second program synthesis benchmark suite. Gecco'21. Association for Computing Machinery, New York, NY, USA. 2021;785-794.
22. Imai S. Is GitHub Copilot a substitute for human pair-programming? An empirical study. Icse'22. Association for Computing Machinery, New York, NY, USA. 2022;319-321.
23. Yetistiren B, Ozsoy I, Tuzun E. Assessing the quality of GitHub Copilot's code generation. Promise 2022. Association for Computing Machinery, New York, NY, USA. 2022;62-71.
24. Sarsa S, Denny P, Hellas A, Leinonen J. Automatic generation of programming exercises and code explanations using large language models. Icer'22. Association for Computing Machinery, New York, NY, USA. 2022;27-43.
25. Finnie-Ansley J, Denny P, Becker BA, Luxton-Reilly A, Prather J. The robots are coming: exploring the implications of OpenAI codex on introductory programming. Ace'22. Association for Computing Machinery, New York, NY, USA. 2022;10-19.
26. Denny P, Kumar V, Giacaman N. Conversing with Copilot: exploring prompt engineering for solving CS1 problems using natural language. Sigcse 2023. Association for Computing Machinery, New York, NY, USA. 2023;1136-1142.
27. Wermelinger M. Using GitHub Copilot to solve simple programming problems. Sigcse 2023. Association for Computing Machinery, New York, NY, USA. 2023;172-178.
28. Brennan RW, Lesage J. Exploring the implications of OpenAI codex on education for industry 4.0. In: Borangiu T, Trentesaux D, Leitão P, eds. *Service Oriented, Holonic and Multi-Agent Manufacturing Systems for Industry of the Future*. Springer International Publishing; 2023:254-266.
29. Munley C, Jarmusch A, Chandrasekaran S. LLM4VV: developing LLM-driven testsuite for compiler validation. *Future Gener Comput Syst*. 2024;160:1-13. doi:10.1016/j.future.2024.05.034
30. Mahmud QI, TehraniJamsaz A, Phan HD, Ahmed NK, Jannesari A. *GNN-Guided Automatic Code Parallelization using Large Language Models*. AUTOPARLLM; 2023.
31. Ding X, Chen L, Emani M, et al. HPC-GPT: integrating large language model for high-performance computing. SC-W'23. Association for Computing Machinery, New York, NY, USA. 2023;951-960.
32. Chen L, Lin PH, Vanderbruggen T, Liao C, Emani M, Supinski B. LM4HPC: towards effective language model application in high-performance computing. In: Mc S, Klemm M, Supinski BR, Deakin T, Klinkenberg J, eds. *OpenMP: Advanced Task-Based, Device and Compiler Programming*. Springer Nature Switzerland; 2023:18-33.
33. Kadosh T, Hasabnis N, Vo VA, et al. Scope Is all you Need: Transforming LLMs for HPC Code. 2023.
34. Nichols D, Marathe A, Menon H, Gamblin T, Bhatele A. HPC-coder: modeling parallel programs using large language models. 2024;1-12.
35. Rasley J, Rajbhandari S, Ruwase O, He Y. DeepSpeed: system optimizations enable training deep learning models with over 100 billion parameters. KDD'20. Association for Computing Machinery, New York, NY, USA. 2020;3505-3506.
36. Lei B, Ding C, Chen L, Lin PH, Liao C. Creating a Dataset for High-Performance Computing Code Translation Using LLMs: A Bridge between OpenMP Fortran and C++. 2023.
37. Chen L, Ahmed NK, Dutta A, et al. The Landscape and Challenges of HPC Research and LLMs. 2024.
38. Godoy W, Valero-Lara P, Teranishi K, Balaprakash P, Vetter J. Evaluation of OpenAI codex for HPC parallel programming models kernel generation. ICPP Workshops'23. Association for Computing Machinery, New York, NY, USA. 2023;136-144.
39. Valero-Lara P, Huante A, Lail MA, et al. Comparing Llama-2 and GPT-3 LLMs for HPC Kernels Generation. 2023.
40. Lu D, Wu J, Sheng Y, Liu P, Yang M. Analysis of the popularity of programming languages in open source software communities. 2020;111-114.
41. OpenACC Architecture Review Board. OpenACC Application Program Interface Version 3.1. 2020.
42. Carter Edwards H, Trott CR, Sunderland D. Kokkos: enabling manycore performance portability through polymorphic memory access patterns. *J Parallel Distrib Comput*. 2014;74(12):3202-3216. doi:10.1016/j.jpdc.2014.07.003
43. AMD. AMD ROCm v5.2 Release. 2022.
44. NVIDIA. The API reference guide for Thrust, the CUDA C++ template library. 2022.
45. Van Der Walt S, Colbert SC, Varoquaux G. The NumPy array: a structure for efficient numerical computation. *Comput Sci Eng*. 2011;13(2):22-30.
46. Lam SK, Pitrou A, Seibert S. Numba: a LLVM-based python JIT compiler. 2015;1-6.
47. Klöckner A, Pinto N, Lee Y, Catanzaro B, Ivanov P, Fasih A. PyCUDA and PyOpenCL: a scripting-based approach to GPU run-time code generation. *Parallel Comput*. 2012;38(3):157-174. doi:10.1016/j.parco.2011.09.001
48. Nishino R, Loomis SHC. Cupy: A Numpy-Compatible Library for Nvidia Gpu Calculations 31st Confernce on Neural Information Processing Systems. 2017;151(7).
49. Knopp T. Experimental multi-threading support for the Julia programming language. IEEE. 2014;1-5.

15320634, 2024, 26, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.8269 by Izmir Yuksek Teknoloji Ensti, Wiley Online Library on [07/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

50. Besard T, Foket C, De Sutter B. Effective extensible programming: unleashing Julia on GPUs. *IEEE Trans Parallel Distrib Syst*. 2019;30(4):827–841. doi:10.1109/TPDS.2018.2872064
51. Samaroo J, Churavy V, Phillips W, et al. JuliaGPU/AMDGPUjl. 2022;v0.4.1.
52. Churavy V, Aluthge D, Wilcox LC, et al. JuliaGPU/KernelAbstractionsjl. 2022;v0.8.3.
53. Jin Z. Hecbench. https://github.com/zjin-lcf/HeCBench. 2023.
54. Jin Z. The Rodinia Benchmarks in SYCL. tech. rep., Oak Ridge National Lab.(ORNL), Oak Ridge, TN (United States). 2021.
55. Klöckner A. pycuda 2022.2.2 documentation. https://documen.tician.de/pycuda/. 2023.
56. Preferred Networks I, Preferred Infrastructure I. CuPy-NumPy & SciPy for GPU. https://docs.cupy.dev/en/stable/. 2023.
57. Kailasa S, Wang T, Barba LA, Betcke T. PyExaFMM: an Exercise in Designing High-Performance Software with Python and Numba. 2023.
58. Godoy WF, Valero-Lara P, Dettling TE, et al. Evaluating performance and portability of high-level programming models: Julia, python/Numba, and Kokkos on exascale nodes. 2023;373-382.
59. Godoy WF, Valero-Lara P, Anderson C, et al. Julia as a unifying end-to-end workflow language on the frontier exascale system. SC-W'23. Association for Computing Machinery, New York, NY, USA. 2023;1989-1999.
60. Churavy V, Godoy WF, Bauer C, et al. Bridging HPC Communities through the Julia Programming Language. 2022.
61. Dongarra J, Graybill R, Harrod W, et al. DARPA's HPCS program: history, models, tools, languages. *Advances in Computers*. Vol 72. Elsevier; 2008:1-100.
62. Dongarra J, Beckman P, Moore T, et al. The international Exascale software project roadmap. *Int J High Perform Comput Appl*. 2011;25(1):3-60. doi:10.1177/1094342010391989
63. Kothe D, Lee S, Qualters I. Exascale computing in the United States. *Comput Sci Eng*. 2019;21(1):17-29. doi:10.1109/MCSE.2018.2875366

How to cite this article: Godoy WF, Valero-Lara P, Teranishi K, Balaprakash P, Vetter JS. Large language model evaluation for high-performance computing software development. *Concurrency Computat Pract Exper*. 2024;36(26):e8269. doi: 10.1002/cpe.8269

APPENDIX

Due to the rapid evolution and statistical nature of these technologies reproducibility and replicability of the present results is a challenging aspect that we expect to improve over time, but has no guarantees as of today. Hence, it's imperative that the reproducibility information is provided.

Kernel generation using GitHub Copilot: https://github.com/keitaTN/Copilot-hpc-kernels.

1. Experiments used the Visual Studio Code GitHub Copilot plugin service on three separate Linux systems using Ubuntu 22.04
2. Experiments were carried between April 14th and April 21st of 2023.
3. The raw dataset are identified by `<kernel>/<language>/<kernel>_outputs.<ext>` directory and file structure.
4. Each Prompt is identified with the line comment

```
Prompt: <kernel> <programming model> <additional text>.
```

Automatic parallelization with ChatGPT: https://github.com/pedrovalerolara/chatgpt-codes.

1. Experiments used the ChatGPT chatbot version 3.5 that can be found here: https://chat.openai.com/.
2. The sequential codes used as part of the prompts can be found in the repository mentioned above. Each programming language has a separate folder for the sequential codes (e.g., https://chatgpt-codes/C/Seq/).
3. The codes generated by ChatGPT were collected in separate folders, one per language (e.g., https://chatgpt-codes/C/3.5/Par/).
4. For those cases where prompt engineering was necessary and provided better results, we included a different folder to store the codes generated (e.g., jacobi_cuda_prompt.cu in https://chatgpt-codes/C/3.5-prompt/Par/).
5. Experiments were carried between October 16 and November 1, 2023.
6. An example of the prompts used for this study is provided in https://chatgpt-codes/prompt.txt.

15320634, 2024, 26, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.8269 by Izmir Yuksek Teknoloji Ensti, Wiley Online Library on [07/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License